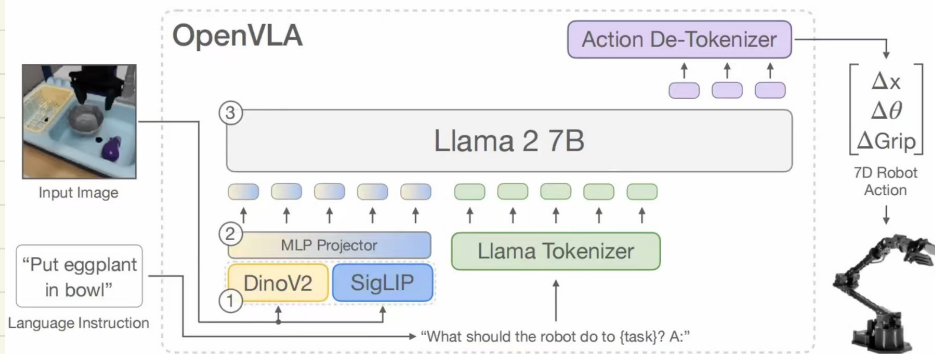


一、OpenVLA的模型架构^[1]



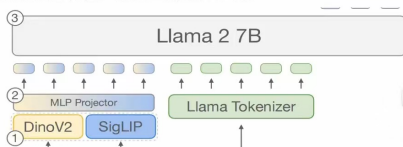
OpenVLA是基于Prismatic-7B (VLM) 作微调 and 扩展的，其由如下构成：

- (1) 语言编码器：Llama Tokenizer。将自然语言分词，映射为索引，然后转化为嵌入向量序列。
- (2) 视觉编码器：使用Dinov2^[3]和SigLIP^[4]分别处理图像的低级空间特征（如纹理、位置、形状等）和高级语义特征（如类别、颜色等）。输入的图像分别经过Dinov2和SigLIP后，输出的特征向量按通道维度进行拼接，即 $(H, W, C1) + (H, W, C2) \rightarrow (H, W, C1 + C2)$ 。

实验表明：与更常用的视觉编码器模块（如仅使用CLIP或者SigLIP）相比，增加Dinov2模块有助于提高LLM的空间推理能力。

选讲Dinov2

- (3) 大语言模型：使用Llama 2 7B负责理解输入的视觉和语言特征，然后生成动作序列。



1. DinoV2 是什么？（它的身世背景）

DinoV2（全称是 Self-Distillation with NO labels）是由 Meta（前 Facebook）发布的一个重量级视觉基础模型。

它最大的特点可以用四个字概括：“自监督学习（Self-Supervised）”。

- 传统的模型（比如 ResNet 或者你圈里的 SigLIP/CLIP）：都是用“图文对”或者“人工打标签”的数据训练出来的。比如给它看一百万张带有“狗”标签的图片，它就学会了认狗。这种模型擅长的是“语义（Semantic）”。
- DinoV2：它在训练时完全没有任何人工标签，也没有文本提示。Meta 喂给了它海量的纯图片，让模型自己跟自己玩（比如遮住图片的一部分让它猜，或者把一张图片切碎了让它找对应关系）。

2. 为什么机器人（OpenVLA）必须要有 DinoV2？

因为像 SigLIP/CLIP 这样的模型，对于机器人来说“眼神不太好”。

我们可以对比一下你截图中的（2）视觉编码器说明：

- **SigLIP（负责高级语义特征）**：你问 SigLIP“杯子在哪？”，它能很快认出画面里哪个是杯子。但是，它眼里的杯子可能只是一个大概的“框”。它不关心杯子的边缘具体在哪、杯把手朝哪个方向、杯子有多深。
- **DinoV2（负责低级空间特征）**：因为 DinoV2 是靠纯图像结构训练出来的，它对图像的几何形状、深度信息、物体边缘、纹理结构极其敏感。哪怕它不知道这个东西叫“杯子”（没有文本概念），它也能极其精准地勾勒出杯把手的轮廓。

机器人抓取，恰恰最需要的就是“空间几何与边缘”！

如果你让机械臂去抓杯子把手，仅仅知道“那是杯子（SigLIP负责）”是不够的，机械臂必须要知道“把手的精确 3D 轮廓和空间位置（DinoV2负责）”，否则夹爪就会撞飞杯子。

3. DinoV2 在 OpenVLA 中的具体工作机制（图解拆解）

结合你的截图，我们看看它俩是怎么配合的：

- **双路并行提取**：当机器人摄像头拍到一张图（比如水槽里的茄子和碗），这张图会同时送给 DinoV2 和 SigLIP。
- **特征拼接 (Concatenation)**：
 - DinoV2 输出了一堆特征张量： $(H, W, C1)$ ，里面装满了诸如“这里有个圆弧边缘”、“那里有个凸起”、“这里的深度很浅”等**空间几何密码**。
 - SigLIP 输出了一堆特征张量： $(H, W, C2)$ ，里面装满了“这是紫色的”、“这是茄子”、“这是塑料碗”等**语言语义密码**。
 - 然后，沿着通道维度把它们像拼积木一样拼在一起： $(H, W, C1) + (H, W, C2) \rightarrow (H, W, C1+C2)$ 。
- **MLP Projector (翻译官)**：拼接后那一大地极其丰富的信息（既有形状又有概念），通过 MLP 投影层，被压缩翻译成了 Llama 2 能看懂的“视觉 Token”。

4. 总结一下 DinoV2 对于 OpenVLA 的意义

在具身智能领域，我们常说一句话：“**CLIP 让你知道抓什么，DINO 让你知道怎么抓。**”

OpenVLA（基于 Prismatic-7B 架构）之所以强大，就是因为它没有像以前的模型那样“只用一个眼睛（单 CLIP）”看世界。引入 DinoV2，相当于给大模型戴上了一副**“3D 几何透视镜”**，弥补了语言大模型天生缺乏空间物理直觉的致命弱点。这就是截图里所说的“提高空间推理能力”的根本原因。

2、动作序列解码器

输入：单张分辨率128*128的图像+语言指令。

Llama 2 7B输出：一组含有终止信号、机器人末端执行器的平移、旋转、夹爪开合程度的动作序列。比如1 128 91 241 5 101 127 217。



编码：

为了使LLM能够直接输出动作参数，本文借鉴RT-2^[5]的做法，将每个动作维度的参数变化范围限定在 $[P_1, P_{99}]$ ，其中第1百分位设为 P_1 ，表示有1%的数据 < P_1 ，然后将 $[P_1, P_{99}]$ 离散为256个箱子（小区间），最后用离散的箱子索引（0,1,...,255）替代连续的动作参数数值。

比如，假设若“夹爪开合程度”这个动作维度的训练数据的取值范围为[0,120mm]，且有1%的数据 < 5，有99%的数据 < 100

则i)夹爪开合程度的取值范围为[5,100]

ii)箱宽 = $(100-5) / 256 \approx 0.37$

iii)若夹爪开合程度 = 5.5，则对应的索引为1

1. 编码 (Encoding)：把“物理动作”翻译成“LLM能懂的词汇”

在训练模型的时候，我们需要把人类遥控器机器人的真实数据教给大模型。

- **为什么要掐头去尾取 $[P_1, P_{99}]$** ：机器人传感器收集的数值往往有噪音（离群点）。比如正常夹爪开合是 0-100mm，但偶尔传感器抽风记录了一个 1000mm。如果按全量最大值来分箱，正常区间就会被极度压缩，导致精度丧失。所以只取 1% 到 99% 的数据，能保证绝大多数动作都能被精细划分。
- **256 个箱子 (Bins)**：就像把一根线段切成了 256 个小格子。每个格子代表一个小动作区间。结合你的图例：夹爪区间 [5, 100]，切成 256 份，每份宽 0.37。如果当前真实动作是 5.5，它落在第 1 个格子里（从0开始算），那么大模型在训练时学到的正确答案就是输出“词汇索引 1”。

2. 解码 (Decoding)：把“LLM吐出的词”还原成“物理动作”

当模型训练好，开始真正控制机器人时，情况反过来了。

- Llama 大脑经过思考，输出了一串 Token 索引，比如图里的 1 128 91 ...。
- 系统拿到索引 1，反向查表：索引 1 对应的箱子范围是 [5.37, 5.74]。
- 为了尽量减少误差，直接取这个箱子的中心值（大概是 5.55）发送给机器人的电机执行。

3. 最绝的工程技巧：怎么塞进 Llama 的词表中？（图二的“注”）

这部分是 OpenVLA 极其聪明的工程实现细节！

- **背景问题**：刚才说了，我们需要 256 个“动作词汇”。如果强行往 Llama 2 的词表里新增 256 个词，那么 Llama 模型最外层的神经网络矩阵维度就变了，这就很难完美继承原版 Llama 2 强大的预训练权重。
- **偷梁换柱大法**：Llama 2 原本有 32000 个词汇。但有些生僻词或乱码（通常排在词表的最后面）是根本用不到的（使用频率极小）。
- **做法**：OpenVLA 的作者干脆“偷点鸭羹”，把 Llama 词表里最后面那 256 个最废柴的 Token（比如什么奇怪的符号组合）直接霸占掉，强行规定：“从今天起，你们这 256 个词不再代表文本，而是代表机器人的 256 个动作箱子！”

总结一下这段的精髓

通过这套“连续动作 -> 百分位截断 -> 256分箱 -> 偷换冷门Token”的丝滑连招，OpenVLA 成功地把“控制物理世界”变成了一个标准的“语言填空游戏”。Llama 2 根本不知道自己在开机器人，它还以为自己在往下接句子。只不过它接的词汇恰好对应了机械臂的空间坐标！

二、训练与实验

1、训练数据与时间（由于DINOv2等模块都是已训好的，故这里训练应该指微调）

训练数据：基于数据集OpenX^[6]（超过2M的机器人轨迹数据）进行筛选，筛选原则与方法为：

（1）确保所有训练数据集输入一致、输出一致。

对此，本文只保留具有第三人称摄像头视角并采用单臂末端执行器控制的操作数据。

（2）确保最终训练混合数据集中包含平衡的机器人形态、任务和场景比例。

对此，本文采用Octo^[7]的数据混合权重策略，对多样性较低的数据集进行降权处理或删除，同时提升包含更多任务和场景多样性的数据集的权重。

（3）过滤掉所有无效或异常的数据，如零值动作。

注：在训练初期，加入了10%的DROIO^[8]。但由于该数据集成功率太低，在训练最后1/3的时间里删除了DROIO，并在其它数据集中重新分配混合权重。

训练时间：使用64张A100，batch size=2048，用时14天。

2、四个实验

所有实验的设置都围绕三个问题来展开：

问题一：在不同机器人平台、环境和任务下测试，OpenVLA与之前的通用机器人策略相比怎么样？

问题二：OpenVLA 是否可以在新的机器人配置和任务上进行高效地微调，并且与最先进的数据高效模仿学习方法相比如何？

问题三：能否使用微调和量化来减少 OpenVLA 模型训练和推理的算力要求？如果可以，如何平衡性能与算力？

实验三：探讨如何高效地参数微调

Strategy	Success Rate	Train Params ($\times 10^6$)	VRAM (batch 16)
Full FT	69.7 ± 7.2 %	7,188.1	163.3 GB*
Last layer only	30.3 ± 6.1 %	465.1	51.4 GB
Frozen vision	47.0 ± 6.9 %	6,760.4	156.2 GB*
Sandwich	62.1 ± 7.9 %	914.2	64.0 GB
LoRA, rank=32	68.2 ± 7.5 %	97.6	59.7 GB
rank=64	68.2 ± 7.8 %	195.2	[†] 60.5 GB

实验三结论：利用LoRA ($r=32$)，我们可以使用一张A100 将OpenVLA微调到新任务上，所需时间为10-15小时。这相比全微调，计算量减少了8倍。